Formalizing Syntax Theory and Derivations in Lean 4

Yu-Lan Dibbits – 6679730

Research Internship Master Artificial Intelligence



Utrecht University

Supervised by: dr. Johannes Korbmacher
Second reader: dr. Dominik Klein

August 25, 2025

Contents

1 An Introduction to Formalizing Formal Logic **2**

2 Lean: the Interactive Proof Assistant **3**

3 Theory **5**
3.1 Notation 5
3.2 Syntax 6
3.2.1 Language 6
3.2.2 Encoding 7
3.3 Proof Theory 9
3.3.1 Derivations 10
3.4 Tarski's Undefinability Theorem 13
3.4.1 The Diagonalization Lemma 13
3.4.2 Equivalence of Transitivity & Inconsistency 13
3.4.3 Formalization 13

4 Conclusions and Future Work **14**

5 Acknowledgements **15**

1 An Introduction to Formalizing Formal Logic

In 1933, Kurt Gödel proved that the class of first-order arithmetic sentences, denoted as $[\exists^* \forall^2 \exists^*, \text{all}, (0)]$, was decidable. The class represents the formulas that begin and end with a certain amount of existential quantifiers and contain exactly two universal quantifiers with no intervening existentials. In the final sentence of his paper, Gödel claimed that the same proof could be extended to the larger class $[\exists^* \forall^2 \exists^*, \text{all}, (0)]=$, which also allows formulas containing an equality predicate. Mathematicians initially accepted Gödel's assertion, and several subsequent results were built on it. However, in the mid-1960s, Stål Aanderaa demonstrated that Gödel's argument does not hold for this extended class, and in 1982 Warren Goldfarb further established that the truth of formulas in the larger class is, in fact, undecidable [3, 9].

Gödel's case highlights a broader issue in proof writing, which has been central to research in both mathematics and formal logic for centuries. The process of doing research by proof writing consists of building a case upon smaller steps of reasoning that can be checked [7]. However, in practice, these smaller steps are often left implicit or glossed over, thus relying on the researchers own intuition and trust of peers. As a result, errors in assumptions can go unnoticed, leading to incorrect conclusions even in widely accepted theorems.

To address this issue, researchers have turned to proof assistants as tools for proof verification. Proof assistants are software tools specifically designed to formalize and prove mathematical theories [21, 17, 7, 6]. Lean is such a tool. More specifically, it is an interactive proof assistant with underlying automated theorem proving features, based on the Calculus of Inductive Constructions. It was developed in 2013 by Leonardo de Moura while employed at Microsoft research, and gained popularity for both its precision and usability [1]. The value of using proof assistants such as Lean is twofold. First, using proof assistants as an integrated tool in research has the effect that it builds trust among researchers. In an academic field such as mathematics or formal logic where the amount of researchers is often scarce and research often relies on human intuition, which can be faulty, academic integrity and validation often relies on trust. Proof assistants allow other academics, also those less involved in the specific area of research, to be able to check the validity of the proofs. This in turn fosters collaboration between researchers that otherwise would not likely have collaborated. Second, formalizing proofs at scale enables the development of AI systems capable of understanding and generating formal mathematics. A key strength of Lean itself is its extensive open-source library, `mathlib`. `Mathlib` is an open source library consisting of both formalized mathematics and useful definitions for programming and was created in 2017 as a community effort by user's of the proof assistant. The library currently contains formalizations of a wide range of fields such as linear algebra, topology, combinatorics, logic and computation and many more. The project is still very active, with many regular contributors and daily activity.

The motivation for this project is to advance the formalization of formal logic which, despite its foundational role in both mathematics and philosophy,

remains underrepresented in the existing proof assistants library. This research focuses specifically on formalizing some of the contents of Volker Halbach's book Axiomatic Theories of Truth. `Mathlib` does contain basic notions such as first-order languages, formulas and theories, but does not contain anything on axiomatic theories of truth nor does it contain some other useful things such as calculus rules and derivations. The next section provides a theoretical introduction as to how Lean operates. Section 3 contains a short overview of the theory of axiomatic theories of truth, as this was the starting point of this research project. Furthermore, the theoretic notions of syntax, derivations and Tarski's undefinability theorem will be discussed, as those are relevant for understanding the formalization work that was done. Moreover, some examples of formalization will be given. And finally, the report concludes with a summary of the remaining work and a reflection on the overall process of formalizing philosophical logic in the Lean theorem prover.

2 Lean: the Interactive Proof Assistant

Formalizing mathematical theories in proof assistants is a difficult process as it often requires someone to work out the smaller steps on which the entire proof is built. As said before, in practice, these smaller steps are often skipped over, creating a considerable gap in the level of detail between written proofs and those that can be checked by a computer. This makes it notably harder to translate a proof into a program. In order to understand how we can formalize mathematical theories, we have to take a look at how Lean operates.

As mentioned earlier, Lean is an interactive proof assistant with underlying automated theorem proving features. It should be noted that there is a difference between proof assistants and automated theorem provers, even though the line between them is often blurred. Automated theorem provers are typically designed to operate within a restricted domain or language and rely on search and decision procedures to automatically generate proofs. Proof assistants, by contrast, are interactive tools that guide users in constructing formal proofs themselves. In practice, many systems, including Lean, combine both approaches to some extent [17]. In Lean, users construct proofs by applying tactics, which are step-by-step instructions that break down complex goals into simpler subgoals. As the proof progresses, Lean constantly updates and displays the remaining goals, ensuring that the proof remains coherent at every step. The automation in Lean arises from its underlying type-checking engine, which incorporates elements of automated theorem proving. Lean's logical foundation gives it a computational interpretation, making it not just a proof assistant, but also a programming language. More precisely, it allows users to write programs with well-defined semantics while also providing a framework for reasoning about those programs. In fact, Lean includes mechanisms for metaprogramming, meaning it can be extended and automated using Lean itself [1].

To truly grasp how proofs are translated into programs in systems like Lean,

two key concepts need to be understood: the Curry-Howard isomorphism and the Calculus of (Inductive) Constructions, which forms the theoretical basis for Lean.

The Curry-Howard isomorphism expresses a correspondence between proof systems and models of computation. More specifically, the correspondence is between systems of formal logic as encountered in proof theory that correspond with computational calculi found in type theory [19]. This idea offers a formalization of the Brouwer–Heyting–Kolmogorov (BHK) interpretation, which views logical formulas as constructions, and proofs as the means by which those constructions are carried out. The BHK interpretation, originating in intuitionistic logic, defines the meaning of logical connectives through the construction of their proofs, as opposed to their truth values [20, 8]. For example, in propositional logic, proving an implication $P \to Q$ corresponds to constructing a function from a value of type P to a value of top Q. Thus, propositions become types, and proofs become lambda terms (i.e. programs).

The BHK interpretation varies depending on how the notion of construction is defined and whether it is viewed as a method for converting one proof to another or for turning an element of a domain into a proof. Different versions of constructivism offer different views on this. By adopting lambda calculus as the defining notion of a construction, the BHK interpretation describes the correspondence between natural deduction and lambda expressions [8].

The Calculus of Constructions (CoC) is a higher-order typed lambda calculus that was developed by Thierry Coquand in the 1980's [4]. It is expressive enough to capture both logical reasoning and computational structure, making it a perfect candidate for formally implementing the Curry-Howard correspondence. This results in the CoC being able to act as both a proof system and a typed programming language that is based on functional programming.

The syntax of terms in the CoC is constructed as follows.

Definition 2.1. Let $\mathbf{T}$ denote the set of all terms.

- $T \in \mathbf{T}$ (the type of all types)
- $P \in \mathbf{T}$ (the type of all propositions)
- Any variable $x \in \mathbf{T}$
- if $A \in \mathbf{T}$and $B \in \mathbf{T}$, then $(AB) \in \mathbf{T}$
- if $A \in \mathbf{T}$, $B \in \mathbf{T}$ and x is a variable, then :
  - $(\forall x : A.B) \in \mathbf{T}$
  - $(\lambda x : A.B) \in \mathbf{T}$

The CoC then consists of five kinds of objects, T, large types, predicates, propositions and proofs. T is T, the type of large types. Large types are the

types of predicates. Predicates are functions that return propositions. Propositions, which are small types and proofs which are terms whose types are propositions. Lean in particular uses the Calculus of Inductive Constructions (CIC), which adds features like inductive types (e.g., natural numbers, lists, trees), recursive definitions, pattern matching and universe hierarchies in order to avoid logical paradoxes [1]. In Lean, writing a proof is equivalent to writing a program whose type is the statement you want to prove. Lean then checks whether that program is well-typed i.e., whether the proof is valid. By interpreting logical connectives and quantifiers as type constructors, and proofs as typed lambda terms, formal logic becomes programmable, and programs become proofs. This is what enables Lean to verify formal proofs, provided that every step is fully and precisely specified.

3 Theory

The ultimate aim of this project was to formalize some parts of Axiomatic Theories of Truth by Volkert Halbach [11]. The main idea of Halbach's research was to conceptualize the notion of truth by axiomatizing the truth predicate in such a way that it corresponds to our intuitive understanding of the definition of truth. In order to achieve this, a few things are needed. In the next section we will discuss some notations in Lean that deviate from the standard conceptualizations of first-order logic. Then we will discuss the implementation of syntax theory as well as the implementation of proof theoretic derivations, which both were needed for this project as well as future proofs. Furthermore we will discuss the set up for the proof of Tarski's Theorem starting with the Diagonal Lemma and discussing the progress made on setting up the actual proof itself.

3.1 Notation

There are some limitations that arise due to Lean's additional functionalities that prevent the code from precisely following the standard conceptualizations of set theory. The first being that the symbol 0 has a reserved meaning in Lean. `Nat.zero` is an inhabitant of the primitive type Nat, which describes the natural numbers. However, as it was necessary to define a language involving the concept of zero, we decided to refer to it as `null` Furthermore, for the implementation of first-order formulas, De Bruijn indices were used [5]. This has resulted in quantifiers not being accompanied by the variable they bind, but instead that variables are natural numbers encoding the position of the quantifier that binds them. For example, a formula in first-order syntax such as

$$\forall x \forall y \forall z (P(x) \vee P(y) \vee P(z) \tag{1}$$

becomes

$$\forall \forall \forall (P(2) \vee P(1) \vee P(0)) \tag{2}$$

3.2 Syntax

We began by formalizing syntax theory, as this was also required for a separate project. Much of the material proved in Halbach's book depends on a formal account of syntax, which involves defining a language $\mathcal{L}$ and an encoding function that maps expressions of $\mathcal{L}$ to their corresponding codes. The following section provides the theoretical background necessary to understand this framework.

3.2.1 Language

From a model theoretic perspective, a Language can be inductively defined as follows [2].

Definition 3.1. A Language or Signature $\mathcal{L}$ consists of

1. a set of constants
2. a set of function symbols, with arity $n \in \mathbb{N}$
3. a set of relation symbols with arity $n \in \mathbb{N}$

It should be noted however, that the set of constants can also be defined as 0-ary functions. This definition without constants does not limit the expressive power of the definition of a language.
Then for terms in a language $\mathcal{L}$, the definition is as follows.

Definition 3.2. Terms are the smalles set of expressions such that

1. constants are terms
2. variables are terms
3. if f is an n-ary function symbol and $t_1, ..., t_n$ are terms, then $f(t_1, ..., t_n)$ is also a term.

A term that does not contain any variables is called a closed term.
The definition of formulas deviates slightly from the conceptual notion that is known from model theory, as `mathlib` contains the concept of a bounded formula. A bounded formula can be denoted by $B(\mathcal{L}, \alpha, n)$ such that $\mathcal{L}$ is a language, α denotes the free variables and n the number of variables that remain to be bound by a quantifier. Then a bounded formula can be defined as follows.

Definition 3.3. The set of bounded formulas is the smallest set of expressions X such that

1. if s and t are terms then $s = t \in X$
2. if P is an n-ary relation symbol and $t_1, ..., t_n$ are terms then $P(t_1, ..., t_n) \in X$
3. if ϕ and ψ are bounded formulas then $\phi \wedge \psi, \phi \vee \psi, \phi \to \psi, \neg\phi \in X$
4. if ϕ is a bounded formula then $\exists\phi, \forall\phi \in X$

3.2.2 Encoding

The axiomatization of syntax consists of encoding expressions of a from a language $\mathcal{L}$ [12]. To achieve that, an encoding function had to be implemented. While I did not implement this function myself, we made use of an existing encoding mechanism from `mathlib` The encoding function that we used, although not identical to Gödel-numbering, shares many similarities. Like Gödel-numbering, the function assigns a unique natural number to each expression, and the encoding is fully reversible. For clarity, let us briefly recall how Gödel-numbering works, assuming we had used it for our own encoding. Gödel-numbering is a method that assigns a unique natural number, called a Gödel number, to each symbol and well-formed formula of a language $\mathcal{L}$. Gödel's original scheme is based on prime factorization. A unique natural number is assigned to each symbol in $\mathcal{L}$. Then a formula, which is a sequence of symbols, is encoded taking the product of the first n primes raised to the power corresponding to the unique code number of each symbol. Since prime factorization is unique, any such number can be decoded unambiguously, allowing for full recovery of the original formula from its Gödel-number [15]. This encoding method can be used in two ways. First, to encode sequences of symbols that represent formulas. And second, to encode sequences of formulas that represent proofs. To denote the Gödel number of an expression ϕ in $\mathcal{L}$, we use the standard notation $\ulcorner\phi\urcorner$. The same notation can also be used inside a formal language such that $\ulcorner\phi\urcorner$ stands for the numeral of the Gödel-code of ϕ [18]. Smith [2007], gives a very neat example, using Peano Arithmetic to illustrate the double use of notation:

1. $SSS0$ is an expression in $\mathcal{L}$, also called the standard numeral for 3
2. Using Gödel's numbering scheme $\ulcorner SSS0\urcorner$, the Gödel-code of $'SS0'$, is $2^3 \cdot 3^3 \cdot 5^3 \cdot 7^1$
3. We can also use the expression $'\ulcorner SSS0\urcorner'$ inside (an extension of) the language $\mathcal{L}$, as an abbreviation for the standard numeral for the Gödel code of $SSS0$ i.e. as an abbreviation for $SSS...S0$ with $2^3 \cdot 3^3 \cdot 5^3 \cdot 7^1$ occurrences of $'S'$.

Before turning to the encoding of terms and expressions in our language, it is helpful to first consider the broader context, namely, the framework of axiomatic theories of truth. To discuss truth formally within arithmetic, we must first distinguish between two levels of language. One of which is called the object language, which we refer to as $\mathcal{L}$ and the other is the meta language, which we refer to as $\mathcal{L}_t$. In this case $\mathcal{L}$ is the language of first-order Peano Arithmetic (PA), which contains symbols for zero, successor, addition and multiplication. Then, by adding a unary truth predicate Tr to $\mathcal{L}$, we obtain $\mathcal{L}_t$. The goal of adding Tr is to reason within arithmetic about which statements of arithmetic are true, effectively internalizing the semantic notion of truth. But to do this rigorously, we must be able to refer to formulas as objects, that is, to treat expressions syntactically within our formal system. Hence, the axiomatization of syntax

concerns itself with defining the notions of formulas, terms, substitution and expressions within the system itself [12].

Within the formal theory, there exists a term $\ulcorner\neg\phi\urcorner$ which denotes the numeral corresponding to the code of ϕ, such that ϕ is a formula in $\mathcal{L}$ and $\ulcorner\neg\phi\urcorner$ is a numeral in $\mathcal{L}_t$. Then, $\neg\ulcorner\phi\urcorner$ is also a term in $\mathcal{L}_t$, denoting the code of $\neg\phi$. It should be noted that $\cdot\neg$ is not the negation as defined in the object language, applied to $\mathcal{L}$, but rahter a function symbol $Neg(n)$ in $\mathcal{L}_t$ denoting the Gödel-number of the negation of the formula with Gödel-number n. We can then state the following, $\cdot\neg\ulcorner\phi\urcorner = \ulcorner\neg\phi\urcorner$, which states that the code that is obtained by applying the negation operation to the code of ϕ is equal to the code of $\neg\phi$. Within the object language, the negation operation is represented by the term $\cdot\neg\phi$, resulting in $\neg\ulcorner\phi\urcorner = \ulcorner\neg\phi\urcorner$ being an arithmetical formula in $\mathcal{L}_t$ [11].

Furthermore, using the encoding function, predicates can be defined such that

1. $Form(x)$ stands for x is the Gödel-code of a well-formed formula
2. $Term(x)$ stands for x is the code of a term
3. $Subst(x, y, z)$, where z codes the result of substituting term y for variable x in formula z

Following Halbach's book, the notions of a closed term i.e. a term with no free variables and the denotation function were also represented in the axiomatization. The denotation function, or as Halbach calls it, the evaluation function, when applied to a number that encodes a closed term, gives the number that term evaluates to [11].

The formalization of the axiomatization of syntax in Lean is then rather straightforward. Below is a snippet of code provided to give an idea of how the logical operators, the substitution function, closed terms and the denotation function are represented in $\mathcal{L}$.

```
def conj_repres (φ ψ : Formula ℒ ℕ): ℒ.Sentence :=
  (⌜φ⌝ ·∧ ⌜ψ⌝) =' (⌜φ ∧' ψ⌝)
def subs_repres (φ : BoundedFormula ℒ ℕ 0) (t : Term ℒ (ℕ ⊕ Fin 0)) :
    ℒ.Sentence :=
  Subs(⌜φ⌝, ⌜(@Term.var ℒ (ℕ ⊕ Fin 0) (.inl 0))⌝, ⌜t⌝) =' ⌜φ/[t]⌝
def closed_term_repres (t : Term ℒ (ℕ ⊕ Fin 0)) : ℒ.Sentence :=
  ClosedTerm(⌜t⌝)
def denote_repres (t : Term ℒ (Empty ⊕ Fin 0)) : ℒ.Sentence :=
  ClosedTerm(⌜t⌝) ⟹ ((·°(⌜t⌝)) =' t)
```

Lastly, it should be noted that for the implementation of the syntax axioms, the axioms were added to the language as functional definitions. There are many ways to go about defining when using `def` in Lean and one way to do so would be by defining something as an abbreviation i.e. defining something using other already existing concepts. Alternatively, we chose to use `def` as a stipulative definition. A stipulative definition assigns a specific, new meaning to a term, often for a particular purpose or to clarify communication. The focus

is mostly on creating a new meaning or narrowing an existing one to a specific context. As a result, such definitions are not a matter of being correct, but rather whether they are deemed useful or not useful depending on the context and the purpose of the definition [10].

3.3 Proof Theory

Furthermore, we have formalized a set of rules and axioms for deriving first-order formulas from the axioms of theories. These rules were essential, both for working out simple derivations and for proving additional results such as Tarski's Theorem and the conservativity of TB. We based our approach on the G3c and G3cp sequent calculus systems, with the initial implementation carried out by someone else (see Acknowledgements). As our work progressed, we found it necessary to introduce additional structural rules, which were incorporated as meta-rules.

In proof theory, deriving a first-order formula from a theory is formalized as constructing a sequent derivation. A derivation is built within a formal system consisting of sequents and inference rules. A sequent has the form $\Gamma \vdash \Delta$ where Γ and Δ are, possibly empty, finite sets of formulas representing, respectively, assumptions and conclusions [13].

A first-order theory can then be described in terms of three components:

1. a formal language $\mathcal{L}$, providing the syntax for formulas,
2. a set formulas in $\mathcal{L}$ that function as axioms. A formula A can be expressed as a sequent of the form $\vdash A$ for any $A \in \mathcal{L}$, and
3. a set of structural and logical rules that dictate how sequents can be transformed to derive new sequents. These include rules for conjunction, disjunction, implication, quantifiers, and structural rules such as weakening, contraction, and cut.

Using these elements, a formula A is derivable from the theory if there exists a finite sequence of applications of these rules that results in the sequent $\vdash A$ [13].

The following set of inference rules formed the basis of our derivations.

Theoretical Axiom if $A \in Th$ then $\Gamma \Rightarrow A, \Delta$

G3cp

Logical Axiom $\Gamma, P \Rightarrow \Delta, P$

Logical Rules

$$\frac{A, B, \Gamma \Rightarrow \Delta}{A \wedge B, \Gamma \Rightarrow \Delta} L\wedge \qquad \frac{\Gamma \Rightarrow \Delta, A \qquad \Gamma \Rightarrow \Delta, B}{\Gamma \Rightarrow \Delta, A \wedge B} R\wedge$$

$$\frac{A, \Gamma \Rightarrow \Delta \qquad B, \Gamma \Rightarrow \Delta}{A \vee B, \Gamma \Rightarrow \Delta} L\vee \qquad \frac{\Gamma \Rightarrow \Delta, A, B}{\Gamma \Rightarrow \Delta, A \vee B} R\vee$$

$$\frac{\Gamma \Rightarrow \Delta, A \qquad B, \Gamma \Rightarrow \Delta}{A \to B, \Gamma \Rightarrow \Delta} L\to$$

$$\frac{}{\bot, \Gamma \Rightarrow \Delta} L\bot \qquad \frac{A, \Gamma \Rightarrow \Delta, B}{\Gamma \Rightarrow \Delta, A \to B} R\to$$

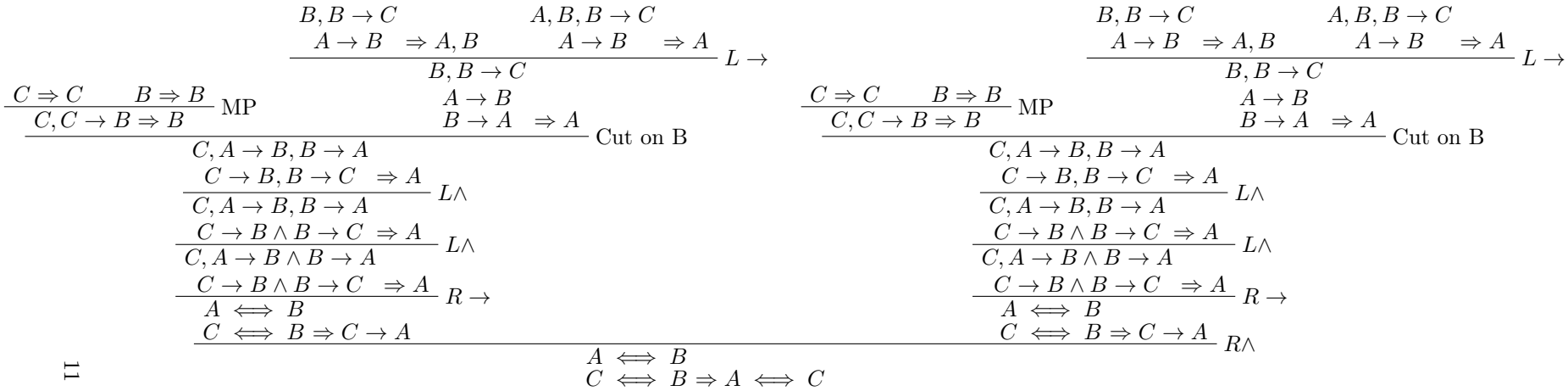
G3c

$$\frac{A(t/x), \forall x A, \Gamma \Rightarrow \Delta}{\forall x A, \Gamma \Rightarrow \Delta} L\forall \qquad \frac{\Gamma \Rightarrow \Delta, A(y/x)}{\Gamma \Rightarrow \Delta, \forall x A} R\forall$$

$$\frac{A(y/x), \Gamma \Rightarrow \Delta}{\exists x A, \Gamma \Rightarrow \Delta} L\exists \qquad \frac{\Gamma \Rightarrow \Delta, \exists x A, A(t/x)}{\Gamma \Rightarrow \Delta, \exists x A} R\exists$$

3.3.1 Derivations

In our formalization, rules such as for example modus ponens, conjunction introduction, and excluded middle are not taken as primitive, but are instead derived from the more basic sequent calculus rules. Proving these derivations is important for two reasons. First, it ensures that the extended rule set we use is admissible in our system, meaning it follows from the base rules without introducing any additional logical strength. Second, in the context of a proof assistant like Lean, having these rules as proven lemmas makes them directly reusable in later proofs, reducing the need to repeatedly carry out the underlying low-level derivations. This both preserves the theoretical soundness of the system and improves the efficiency of formal work. Below is a proof tree showing the equivalence of transitivity (see Tarski's Undefinability Theorem).


B, B → C
A → B ⇒ A, B
A, B, B → C
A → B ⇒ A
L →
B, B → C
C ⇒ C
B ⇒ B
MP
C, C → B ⇒ B
A → B
B → A ⇒ A
Cut on B
C, A → B, B → A
C → B, B → C ⇒ A
L∧
C, A → B, B → A
C → B ∧ B → C ⇒ A
L∧
C, A → B ∧ B → A
C → B ∧ B → C ⇒ A
R →
A ⟺ B
C ⟺ B ⇒ C → A
R∧
A ⟺ B
C ⟺ B ⇒ A ⟺ C

The code that is provided below follows the proof tree with the first part following the left side and the second part following the right side.

```
lemma eqv_trans : ∀Th : L.Theory, ∀(A B C : L.Formula ℕ), Nonempty
    (Derivation Th {A ⇔ B, C ⇔ B} {A ⇔ C}) := by
  let eqv_trans_derivation
    (Th : L.Theory) (A B C : Formula L ℕ) :
    Derivation Th {A ⇔ B, C ⇔ B} {A ⇔ C} := by
    dsimp [FirstOrder.Language.BoundedFormula.iff]
    dsimp [instMin]
    apply Derivation.right_conjunction (A ⟹ C) (C ⟹ A) {A ⟹ C} {C
    ⟹ A} ∅
    apply Derivation.right_implication A C {A, (A ⟹ B) ⊓ (B ⟹ A),
    (C ⟹ B) ⊓ (B ⟹ C)} {C} ∅
    apply Derivation.left_conjunction (A ⟹ B) (B ⟹ A) {A, (A ⟹
    B), (B ⟹ A), (C ⟹ B) ⊓ (B ⟹ C)} {A, (C ⟹ B) ⊓ (B ⟹ C)}
    apply Derivation.left_conjunction (C ⟹ B) (B ⟹ C) {A, (A ⟹
    B), (B ⟹ A), (C ⟹ B), (B ⟹ C)} {A, A ⟹ B, B ⟹ A}
    apply Calculus.cut B {A, (A ⟹ B)} ∅ {(B ⟹ A), (C ⟹ B), (B ⟹
    C)} {C}
    apply mp_derivation
    rw [← Finset.insert_eq]
    apply Derivation.left_implication B C {B, (B ⟹ A), (C ⟹ B)} {C,
    B} {C, B, (B ⟹ A), (C ⟹ B)}
    apply Derivation.lax
    simp
    ...
    apply Derivation.right_implication C A {C, (A ⟹ B) ⊓ (B ⟹ A),
    (C ⟹ B) ⊓ (B ⟹ C)} {A} ∅
    apply Derivation.left_conjunction (A ⟹ B) (B ⟹ A) {C, (A ⟹
    B), (B ⟹ A), (C ⟹ B) ⊓ (B ⟹ C)} {C, (C ⟹ B) ⊓ (B ⟹ C)}
    apply Derivation.left_conjunction (C ⟹ B) (B ⟹ C) {C, (C ⟹
    B), (A ⟹ B), (B ⟹ A),  (B ⟹ C)} {C, A ⟹ B, B ⟹ A}
    apply Calculus.cut B {C, (C ⟹ B)} ∅ {(A ⟹ B), (B ⟹ A), (B ⟹
    C)} {A}
    apply mp_derivation
    rw [← Finset.insert_eq]
    apply Derivation.left_implication B A {B, (A ⟹ B), (B ⟹ C)} {A,
    B} {A, B, (A ⟹ B), (B ⟹ C)}
    apply Derivation.lax
    simp
    rw [Finset.insert_eq]
    apply Derivation.lax
    simp
    ...
  intro Th A B C
  apply eqv_trans_derivation at Th
  apply Th at A
  apply A at B
  apply B at C
```

```
apply Nonempty.intro C
```

3.4 Tarski's Undefinability Theorem

At this stage, we turned our attention to Tarski's undefinability theorem, since many of the components required for its formalization were already in place. In short, Tarski's undefinability theorem shows that the encoding of expressions cannot be executed for semantic concepts such as truth, i.e. no theory containing a sufficiently interpreted language can represent its own semantics.

The formal proof goes as follows. Let $T(\ulcorner\psi\urcorner)$ denote a formula that is true in the interpretation of $\mathcal{L}$. Then for all ψ, $T(\ulcorner\psi\urcorner) \iff \psi$ holds in the interpretation. Let there be a sentence ψ such that it is written in the language of the theory Th. Then we can prove that $Th \vdash (\psi \iff \neg T(\ulcorner\psi\urcorner))$ by applying the diagonal lemma to a formula ϕ with one free variable x, such that $\phi = \neg T(x)$. This leads to a contradiction [14].

It follows that certain parts of the proof still require formalization, namely, the diagonalization lemma, the equivalence of transitivity lemma, and the inconsistency lemma, which will be discussed below.

3.4.1 The Diagonalization Lemma

The diagonalization lemma states the following. If we have a first-order theory in the language of arithmetic, and that theory also contains the arithmetic itself. Let $\phi(x)$ be a formula in the language of $\mathcal{L}$ with only one free variable, x. Then there exists a sentence ψ, in $\mathcal{L}$, such that T$\vdash \psi \iff \phi(\ulcorner\psi\urcorner)$. One might be able to see then that ψ represents a self-referential sentence, referring to something along the lines of "This sentence has the property ϕ". Now let that ϕ be the property of the negated truth-predicate, thus creating a sentence analogous to "This sentence is false" [16].

3.4.2 Equivalence of Transitivity & Inconsistency

The equivalence of transitivity states that $A \iff B$ and $C \iff B$, then $A \iff C$. Applying this to Tarski's undefinability theorem allows us to prove that if $T(\ulcorner\phi\urcorner) \iff \phi$ and $\phi \iff \neg T(\ulcorner\phi\urcorner)$, then $T(\ulcorner\phi\urcorner) \iff \neg T(\ulcorner\phi\urcorner)$. Moreover, proving inconsistency shows that $A \iff \neg A$, which allows us to prove $T(\ulcorner\phi\urcorner) \iff \neg T(\ulcorner\phi\urcorner)$. Thus providing everything we need to prove Tarski's undefinability theorem.

3.4.3 Formalization

Below, we provide the formalization of the diagonalization lemma.

```
axiom diagonal_lemma
  (φ : BoundedFormula ℒ␣ (Fin 1) 0) :
  ∃ (ψ : Sentence ℒ␣),
    TB ⊢
```

```
(bf_empty_to_bf_N (ψ ⇔ (Induction.formula_substitution (⌜ψ⌝) φ)))
```

The implementation of the substitution function was carried out by someone else (see Acknowledgements). The formalization of Tarski's Undefinability Theorem remains unfinished as handling the substitution function and proving the remaining goals continues to be challenging. Formalizing the inconsistency lemma follows similar steps as the equivalence of transitivity. For further details, the project is available on GitHub.

4 Conclusions and Future Work

The main goal of this project was to explore the potential of using the Lean 4 interactive theorem prover for the formalization of results from philosophical logic, with a focus on axiomatic theories of truth. This area remains largely unformalized, making it an interesting case for seeing how such tools can be applied. We set up our formalizations so they could be reused for other results in Halbach's Axiomatic Theories of Truth, while keeping the verification accessible to those who are not familiar with type-theoretic notions. Over the course of the project, we successfully formalized some notable basic concepts required for the formalization of Axiomatic Theories of Truth, including the axiomatization of Syntax Theory as well as some derivations.

These results illustrate that Lean can be a useful tool for formalizing concepts and proofs from Axiomatic Theories of Truth. However, the work also revealed several challenges. First of all, collaborating in a shared repository required careful coordination, as different versions of the same file often diverged. Furthermore, determining the best way to structure the formalization for future extensions as well as the formalization proofs and concepts itself was not always straightforward. This has resulted in some aspects of the project to remain unfinished.

The biggest tasks that remain are the development of a workable approach to the substitution function and completing the implementation of the proof of Tarski's Theorem. The substitution function in particular highlights the difficulties that arise in structuring a formalization. It is generally best to implement concepts as broadly as possible, to facilitate reuse in future projects. However, achieving this is not always straightforward, and in many cases, it only becomes apparent in hindsight that the structure should have been approached differently. Moreover, progress is often hindered by the limited automated reasoning support available for the formalization of formal logic.

Taken together, these experiences suggest that `mathlib` is not yet as developed for formalized formal logic as it is for mathematics, which forces much of the work to rely on ad-hoc implementation. A natural next step would be to extend the library with more general tools for formal logic, both to support future work in this area and to open new possibilities for the use of formalization in AI. Such developments could also encourage a research practice that combines blueprints with Lean code, offering a way to minimize mistakes and oversights,

such as the one that even Gödel himself once made, by ensuring that each step is rigorously checked.

5 Acknowledgements

This was a fun project to do as part of the AI programme at Utrecht University. I would like to thank the following people for their support.

More specifically, I would like to thank Bram Swaanen for writing the encoding part, the substitution function and for implementing the calculus rules. Thanks for the collaboration and I wish you all the best with whatever lies ahead of you.

Furthemore, I would like to thank Sven Holtrop, without whom I'd still be stuck trying to make Lean run on Windows. Thank you for enduring my many frustrations when programming.

And last but not least, I would like to thank my supervisor Johannes Korbmacher for being a great supervisor. Thanks for giving me the opportunity to do this research project in the first place. I have learned a great deal and I am looking forward to learning more.

References

[1] Jeremy Avigad et al. *Theorem Proving in Lean 4*. URL: https://leanprover.github.io/theorem_proving_in_lean4/.

[2] Benno van den Berg. *Syllabus Model Theory 2021/2022*. 2021.

[3] Egon Boerger, Erich Grädel, and Yuri Gurevich. *The Classical Decision Problem*. Springer, 1997.

[4] Thierry Coquand and Huet Gérard. "The Calculus of Constructions". In: *Information and Computation* 76.2 (1988), pp. 95–120. DOI: 10.1016/0890-5401(88)90005-3.

[5] N.G de Bruijn. "Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church-Rosser theorem". In: *Indagationes Mathematicae (Proceedings)* 75.5 (1972), pp. 381–392. DOI: https://doi.org/10.1016/1385-7258(72)90034-0.

[6] Herman Geuvers. *Can the computer really help us to prove theorems?* Veldhoven, The Netherlands, 2011. URL: https://www.cs.ru.nl/~herman/ictopen.pdf.

[7] Herman Geuvers. "Proof Assistants: History, ideas and future". In: *Sadhana* 34 (July 2009), pp. 3–25. DOI: https://doi.org/10.1007/s12046-009-0001-5.

[8] Herman Geuvers. *The Calculus of Constructions and Higher Order Logic*. Tech. rep. Toernooiveld 1, 6525 ED Nijmegen, The Netherlands: University of Nijmegen, 1992.

[9] Warren Goldfarb. *Kurt Gödel: Collected Works. Vol. 1*. Oxford University Press, 1986.

[10] Anil Gupta and Stephen Mackereth. *Definitions*. Aug. 2008. URL: https://plato.stanford.edu/entries/definitions/.

[11] Volker Halbach. *Axiomatic Theories of Truth*. Cambridge University Press, 2014. DOI: https://doi.org/10.1017/CBO9781139696586.

[12] Graham E. Leigh and Carlo Nicolai. "Axiomatic Truth, Syntax and Metatheoretic Reasoning". In: *The Review of Symbolic Logic* 6.4 (Dec. 2013), pp. 613–636. DOI: https://doi.org/10.1017/S1755020313000233.

[13] Sara Negri and Jan von Plato. *Structural Proof Theory*. Cambridge University Press, 2001. DOI: https://doi.org/10.1017/CBO9780511527340.

[14] Panu Raatikainen. *Gödel's Incompleteness Theorem*. Aug. 2020. URL: https://plato.stanford.edu/entries/goedel-incompleteness/#Rel.

[15] Panu Raatikainen. *Supplement: Gödel Numbering*. Aug. 2020. URL: https://plato.stanford.edu/entries/goedel-incompleteness/sup1.html.

[16] Panu Raatikainen. *Supplement: The Diagonalization Lemma*. Aug. 2020. URL: https://plato.stanford.edu/entries/goedel-incompleteness/sup2.html.

[17] Patrick Schnider. *An Introduction to Proof Assistants*. Tech. rep. ETH Zürich, 2014.

[18] Peter Smith. *An Introduction to Gödel's Theorems*. Cambridge University Press, 2007.

[19] Morten Heine B. Sørensen and Paweł Urzyczyn. "Lectures on the Curry-Howard Isomorphism". In: *Elsevier Science* 149.1 (Sept. 2006). DOI: https://doi.org/10.1016/S0049-237X(06)X8001-1.

[20] Anne S. Troelstra. *History of Constructivism in the 20Th Century Vol. Ml-91-05*. University of Amsterdam, 1991.

[21] Freek Wiedijk. "Comparing Mathematical Provers". In: *Proceedings of the Second International Conference on Mathematical Knowledge Management*. MKM '03. Berlin, Heidelberg: Springer-Verlag, 2003, pp. 188–202. ISBN: 3540005684. DOI: https://doi.org/10.1007/3-540-36469-2_15.